

Fairness Without Exposure: Privacy-Preserving Phragmén Voting

Liran Shem-Tov^a, Tamir Tassa^a, Erel Segal-Halevi^b

^a*The Open University of Israel*

^b*Ariel University*

Abstract

@@@

1. Introduction

Consider a group of people who need to elect a president repeatedly each year. Suppose that 60% of the voters support candidate A and the other 40% support candidate B. In a single year, any reasonable voting rule would elect candidate A. But if the same vote is repeated each year, candidate A will be the president in 100% of the years. This is arguably unfair towards the 40% of the population who support candidate B. To guarantee that all sectors in the population are represented, we would like each cohesive group of voters to be able to determine the winner in a fraction of years proportional to its size; in the example, candidate B should be elected in 40% of the years.

In reality, there may be more than two candidates, and the voters' preferences may be more complex: each voter may approve any subset of candidates. There are various methods that can guarantee different “fair representation” properties even in this more complex setting. One such method is Phragmén's sequential rule [?]. Informally, the method works as follows. Each voter has a “virtual wallet” which is initially empty. We continuously add virtual money to the wallet of each voter, in an equal speed. The “cost” of each candidate is set to 1. Whenever the supporters of a certain candidate have together a total sum equal to the candidate's cost, they “buy” the candidate — it is elected, and the supporters' wallets are emptied. The voters

Email addresses: liranst13@gmail.com (Liran Shem-Tov), tamirta@openu.ac.il (Tamir Tassa), erelsgl@gmail.com (Erel Segal-Halevi)

who do not support the elected candidate keep their wallet; this effectively gives them an advantage in next year's elections. One can easily check that, in the initial example, Phragmén's rule would indeed elect candidate B in about 40% of the years.

But Phragmén's rule fails on another consideration, which is very important in elections: privacy. Most election systems in democratic countries spend a lot of effort in guaranteeing complete privacy and anonymity of votes: voters go behind a curtain, put their vote in an unmarked envelope, and put the envelope in a box with all other envelopes, such that it is impossible to know which voter cast which vote. In contrast, Phragmén's rule must know which voter cast which vote, in order to maintain the voters' wallets for the next year. This problem is not particular to Phragmén's rule: any voting rule that guarantees any form of fairness over the years, must keep some "memory" of past votes.

We propose to solve this conflict using *secure multi-party computation*.

2. Related Work

See <https://www.jair.org/index.php/jair/article/view/18660> — sequential voting.

See <https://link.springer.com/article/10.1007/s10107-023-01926-8> — numeric issues in computing Phragmen's rule.

See <https://link.springer.com/article/10.1007/s10058-025-00385-1> - robustness of voting rules to random noise; may be related to numeric stability.

See <https://link.springer.com/article/10.1007/s00355-023-01498-8> - dynamic proportional ranking.

3. Phragmén's Sequential Rule

We consider a setting in which a population of voters periodically elects an officeholder from a given set of candidates. The purpose of Phragmén's sequential rule is to provide a voting mechanism that takes into account the outcomes of previous elections when determining the winner of the current election, thereby promoting fair long-term representation of the voters' preferences.

Preliminaries. In each election period there is a set of N voters, $\mathbf{V} = \{V_1, \dots, V_N\}$, and a set of M candidates, $\mathbf{C} = \{C_1, \dots, C_M\}$. Each voter V_n , $n \in [N]$, submits an approval ballot $\mathbf{b}_n \in \{0, 1\}^M$ whose 1-entries indicate the approved candidates. The goal in each election period is to select a single winner $C_m \in \mathbf{C}$.

For readability, we use feminine pronouns when referring to candidates and masculine pronouns when referring to voters.

The selection rule in a given period. Phragmén's sequential rule may be viewed as a process in which each voter maintains a virtual wallet that accumulates budget over time.

For each voter V_n , let $w_n \in \mathbb{R}_{\geq 0}$ denote the amount of budget in V_n 's wallet at the start of the current period. At the beginning of the first period, all wallets are initialized to zero:

$$w_n \leftarrow 0 \quad \text{for all } n \in [N].$$

However, at the start of later periods, some voters may have a positive balance in their wallet, retained from previous periods.

Each candidate has unit cost. A candidate C_m is said to be *fundable* if the total budget currently held by her approvers is at least 1. Namely, letting

$$A(m) := \{n \in [N] : \mathbf{b}_n(m) = 1\}$$

denote the set of voters who approve C_m , candidate C_m is fundable if and only if

$$\sum_{n \in A(m)} w_n \geq 1.$$

Intuitively, a candidate C_m becomes fundable once the voters who approve her can jointly pay her unit cost from their current wallets.

If at the start of the current period some candidate is already fundable, then one such candidate is elected arbitrarily. Otherwise, no candidate is fundable initially, and the rule increases the wallets of all voters continuously and at identical rates until some candidate becomes fundable.

We proceed to compute, for each candidate C_m , $m \in [M]$, the time until she becomes fundable. Candidate C_m 's approvers jointly require an additional amount of

$$1 - \sum_{n \in A(m)} w_n$$

in order for C_m to become fundable. Since all voters accumulate budget at identical rates, each approver of C_m must accumulate an additional amount of

$$\frac{1 - \sum_{n \in A(m)} w_n}{|A(m)|}.$$

Hence, the elected candidate is one for whom this quantity is minimized:

$$m^* \in \arg \min_{m \in [M]} \frac{1 - \sum_{n \in A(m)} w_n}{|A(m)|}. \quad (1)$$

If there is a tie, then one such candidate is elected arbitrarily.

At the end of the current period, the wallets of all voters who approve the elected candidate C_{m^*} are reset to 0, while the wallets of all other voters increase by an amount equal to the minimal value in Equation (1):

$$w_n \leftarrow \begin{cases} 0, & \text{if } n \in A(m^*), \\ w_n + \frac{1 - \sum_{n' \in A(m^*)} w_{n'}}{|A(m^*)|}, & \text{if } n \notin A(m^*). \end{cases} \quad (2)$$

At each subsequent period, the voters' wallets are initialized using their balances at the end of the previous period. Consequently, voters whose approved candidates were not elected accumulate larger wallet balances over time, making their approved candidates more likely to be elected in future periods.

Observe that in the first period, since all wallets are initially zero, the winner is a candidate with the largest number of approvers.

4. Cryptographic Background

Here we briefly review the two fundamental cryptographic techniques underpinning our protocol—threshold secret sharing (Section 4.1) and multiparty computation (MPC) over secret shares (Section 4.2)—and describe the security model and formal privacy guarantee.

4.1. Threshold Secret Sharing

Secret sharing schemes [?] are protocols that enable distributing a secret scalar s among a set of parties, $P_1, \dots, P_D$. Each party P_d , $d \in [D] := \{1, \dots, D\}$, receives a value $[[s]]_d$, called a *share*, such that certain subsets of shares enable reconstruction of s , while any other subset reveals no information about s . In the basic *threshold* setting, there is a threshold $t \leq D$ such that a set of shares enables reconstruction if and only if its size is at least t .

Shamir’s t -out-of- D threshold secret sharing scheme [?] operates over a finite field $\mathbb{F}_p$, where p is a prime chosen large enough so that all secrets and intermediate values used by the protocol can be represented in $\mathbb{F}_p$. It has two procedures—**Share** and **Reconstruct**:

- **Share _{t,D} (s)**. Sample a uniformly random polynomial $f(\cdot)$ over $\mathbb{F}_p$ of degree at most $t-1$ with constant term $f(0) = s$, and output D shares $[[s]]_d = f(d)$ for $d \in [D]$. The entire set of shares, denoted $[[s]] := \{[[s]]_d : d \in [D]\}$, is called a (t, D) -sharing of s .
- **Reconstruct _{t} ($[[s]]$)**. Given any selection of t shares from the (t, D) -sharing $[[s]]$, interpolate the unique degree- $(t-1)$ polynomial consistent with the selected points and output $s = f(0)$. Any selection of t shares reconstructs s , while any selection of at most $t-1$ shares reveals no information about s .

Hereinafter, we set the threshold to be

$$t := \lfloor (D+1)/2 \rfloor. \quad (3)$$

Namely, reconstructing a secret requires at least t parties, i.e., a strict majority of the D parties. Consequently, as long as at most $t-1$ parties collude, they learn nothing about any shared secret.

4.2. Multiparty Computation Over Secret Shares

Let u and v be two secret values in $\mathbb{F}_p$, and assume that $P_1, \dots, P_D$ hold (t, D) -sharings of them, denoted $[[u]] = \{[[u]]_d : d \in [D]\}$ and $[[v]] = \{[[v]]_d : d \in [D]\}$. Given a publicly known function $G(\cdot, \cdot)$, the goal is to compute a (t, D) -sharing of $G(u, v)$ while revealing no information about u , v , or $G(u, v)$.

We focus on basic building blocks that suffice for expressing our protocol.

Affine combinations. Let $\alpha, \beta, \gamma \in \mathbb{F}_p$ be public. Then

$$[[\alpha]] + \beta[[u]] + \gamma[[v]] := \{\alpha + \beta[[u]]_d + \gamma[[v]]_d : d \in [D]\}$$

is a valid (t, D) -sharing of $w := \alpha + \beta u + \gamma v$, by the linearity of Shamir secret sharing. By writing $[[w]] \leftarrow [[\alpha]] + \beta[[u]] + \gamma[[v]]$ we mean that each party P_d locally sets $[[w]]_d \leftarrow \alpha + \beta[[u]]_d + \gamma[[v]]_d$. In particular, any public constant $\alpha \in \mathbb{F}_p$ can be represented as a sharing $\{[[\alpha]]_d = \alpha : d \in [D]\}$.

Multiplications. A secure multiplication protocol,

$$[[w]] \leftarrow \text{SecureMult}([u], [v]),$$

takes (t, D) -sharings of u and v and outputs a (t, D) -sharing of $w = u \cdot v$ without revealing any information on u , v , or w . [?] designed such a protocol, later improved by [?].

Comparisons. A secure comparison protocol,

$$[[w]] \leftarrow \text{SecureLT}([u], [v]),$$

takes (t, D) -sharings of two secret integers u and v (both smaller than $p - 1$) and outputs a (t, D) -sharing of $w = 1_{u < v}$ [?]. When we compare secret values, we always interpret them as integers in a known public range that is small enough to avoid wrap-around modulo p .

Equality. A secure equality-testing protocol,

$$[[w]] \leftarrow \text{SecureIsZero}([u]),$$

takes a (t, D) -sharing of u and outputs a (t, D) -sharing of $w = 1_{u=0}$ [?].

5. A Privacy-Preserving Implementation of Phragmén Voting

Here we present the protocol PP-PHRAGMÉN, which securely implements Phragmén voting while keeping secret the approval ballots, candidate scores, tie-breaking decisions, and the order in which candidates are selected.

Throughout this paper, we assume that in each election period, at least one candidate receives at least one approval vote; for further details regarding this assumption and the treatment of individual zero-support candidates.

The protocol is executed by a set of *parties* $\mathcal{P} = \{P_1, \dots, P_D\}$. It operates periodically, meaning the parties retain the state of the voters' wallets from one election period to the next.

Notations & Data Representation. Let the accumulated ballots be represented as an $N \times M$ matrix B , where $B_{n,m} = 1$ if and only if voter V_n approves candidate C_m .

A naive approach to representing a voter's wallet w_n would be to store a shared numerator w_n^{num} and denominator w_n^{den} separately. Computing the sum of approvers' wallets for candidate m ,

$$\sum_{n \in A(m)} w_n = \sum_{n \in A(m)} \frac{w_n^{\text{num}}}{w_n^{\text{den}}}$$

would then require a complex loop over the voters, iteratively cross-multiplying shares into a running sum variable σ .

This approach necessitates $O(N)$ sequential **SecureMult** operations per candidate, leading to an impractical exponential depth in the MPC circuit.

To eliminate division and cross-multiplication from the critical path, we employ a **common-denominator scaling strategy**.

We maintain a single secret-shared global denominator $\llbracket \Delta \rrbracket$ and individual scaled wallets $\llbracket \tilde{W}_n \rrbracket$ for each voter, such that the actual wallet balance is mathematically represented as $w_n = \tilde{W}_n / \Delta$ at all times.

Similarly, when determining the time at which candidate C_m becomes fundable, we maintain the score as a shared pair $(\llbracket \text{Score}_m^{\text{num}} \rrbracket, \llbracket \text{Score}_m^{\text{den}} \rrbracket)$.

5.1. Oblivious Candidate Permutation

In the formal definition of Phragmén's rule, ties are typically broken in favor of candidates with smaller indices. While well-defined, this introduces a systematic bias dependent on the public ordering of the candidates. To eliminate this dependence, PP-PHRAGMÉN generates a uniformly random

permutation of the candidates kept secret from the voters, and executes the rule with respect to this internal ordering.

Algorithm 1: Oblivious Candidate Permutation (Run once at setup).

Input: Security parameter λ ; number of candidates M ; number of talliers D .

Output: A public permutation $\pi \in S_M$ of the candidate indices.

```

1 for  $d \leftarrow 1$  to  $D$  do
2    $P_d$  samples  $s_d \leftarrow \{0, 1\}^\lambda$ ;
3    $P_d$  broadcasts  $\text{com}_d \leftarrow \text{SHA-256}(s_d \parallel d)$ ;
4 for  $d \leftarrow 1$  to  $D$  do
5    $P_d$  broadcasts  $s_d$ ;
6   for  $d' \leftarrow 1$  to  $D$ ,  $d' \neq d$  do
7     if  $\text{SHA-256}(s_d \parallel d) \neq \text{com}_d$  then
8       abort;
  // Derive shared seed and permutation locally
9  $s \leftarrow s_1 \oplus s_2 \oplus \dots \oplus s_D$ ;
10  $\pi \leftarrow \text{FisherYates}(s, M)$ ;
11 return  $\pi$ ;
```

5.2. Input Validation, Sanitization & Compaction

Algorithm 2: Input Validation, Sanitization & Compaction.

Input: (t, D) -sharings $\llbracket B_{n,m} \rrbracket$ for all $(n, m) \in [N] \times [M]$.
Output: Compact shared ballot matrix $\{\llbracket \hat{B}_{i,m} \rrbracket\}_{i \in [N_{\text{valid}}], m \in [M]}$, and N_{valid} .

```

1 Cheaters  $\leftarrow \emptyset$ ;
2 for  $n \leftarrow 1$  to  $N$  do
3   for  $m \leftarrow 1$  to  $M$  do
4      $\llbracket u_{n,m} \rrbracket \leftarrow \text{SecureMult}(\llbracket B_{n,m} \rrbracket, \llbracket B_{n,m} \rrbracket - \llbracket 1 \rrbracket)$  ;
        //  $B_{n,m}(B_{n,m} - 1) = 0$  iff  $B_{n,m} \in \{0, 1\}$ 
5      $u_{n,m} \leftarrow \text{Reconstruct}_t(\llbracket u_{n,m} \rrbracket)$ ;
6     if  $u_{n,m} \neq 0$  then
7       Cheaters  $\leftarrow$  Cheaters  $\cup \{n\}$ ;
8     break;
9  $N_{\text{valid}} \leftarrow N - |\text{Cheaters}|$ ;
10  $i \leftarrow 0$ ;
11 for  $n \leftarrow 1$  to  $N$  do
12   if  $n \notin \text{Cheaters}$  then
13      $i \leftarrow i + 1$ ;
14     for  $m \leftarrow 1$  to  $M$  do
15        $\llbracket \hat{B}_{i,m} \rrbracket \leftarrow \llbracket B_{n,m} \rrbracket$ ;
16 return  $(\{\llbracket \hat{B}_{i,m} \rrbracket\}_{i \in [N_{\text{valid}}], m \in [M]}, N_{\text{valid}})$ ;
```

5.3. Initialization

At the start of the first period, the global denominator is initialized to 1, and all scaled wallets are initialized to 0. If this is not the first round, you must claim according to the previous round.

Algorithm 3: Initialization (Run prior to the first period only).

```

1  $\llbracket \Delta \rrbracket \leftarrow \llbracket 1 \rrbracket$ ;
2 for  $i \leftarrow 1$  to  $N$  do
3    $\llbracket \tilde{W}_i \rrbracket \leftarrow \llbracket 0 \rrbracket$ ;
```

5.4. Per-Candidate Score Computation

The use of a global denominator removes division and per-voter cross-multiplication when *summing* the supporters' wallets; comparing two candi-

dates' scores still requires a single integer cross-multiplication (Algorithms 5–6).

The score for candidate m is directly computed as $(\Delta - \tilde{T}_m)/(\Delta \cdot |A(m)|)$, where $\tilde{T}_m$ is the sum of the scaled wallets of her supporters, and $|A(m)|$ is the total supporter count.

Throughout the protocol, every voter V_n satisfies the invariant $w_n = \tilde{W}_n/\Delta$, where $\tilde{W}_n \in \mathbb{F}_p$ is the *scaled wallet* of voter n and $\Delta \in \mathbb{F}_p$ is a global denominator shared by all voters. The tilde marks $\tilde{W}_n$ as a rescaled integer representation of the rational balance w_n , rather than a new quantity.

We derive the score of candidate C_m from Equation (1) as follows. *Step 1.* Substituting the invariant into the supporter sum gives

$$\sum_{n \in A(m)} w_n = \sum_{n \in A(m)} \frac{\tilde{W}_n}{\Delta}.$$

Step 2. Since all supporters share the same denominator Δ , it factors out of the sum:

$$\sum_{n \in A(m)} \frac{\tilde{W}_n}{\Delta} = \frac{1}{\Delta} \sum_{n \in A(m)} \tilde{W}_n = \frac{\tilde{T}_m}{\Delta}, \quad \text{where } \tilde{T}_m := \sum_{n \in A(m)} \tilde{W}_n.$$

Step 3. Inserting this into Equation (1) yields

$$\text{Score}_m = \frac{1 - \tilde{T}_m/\Delta}{|A(m)|}.$$

Step 4. Multiplying numerator and denominator by Δ clears the inner fraction:

$$\text{Score}_m = \frac{\Delta - \tilde{T}_m}{|A(m)| \cdot \Delta}.$$

The protocol therefore represents the score as the numerator/denominator pair

$$\boxed{\text{Score}_m^{\text{num}} = \Delta - \tilde{T}_m, \quad \text{Score}_m^{\text{den}} = |A(m)| \cdot \Delta} \quad (4)$$

This representation involves no division: the numerator is a linear combination of shared values, while the denominator requires only a single **SecureMult** of Δ and $|A(m)|$, with $|A(m)|$ and $\tilde{T}_m$ accumulated at linear cost. In contrast, a per-voter-denominator representation would require an oblivious product over all supporter denominators.

Algorithm 4: Oblivious Score Computation (Global Denominator).

Input: Shared ballot matrix $\{\llbracket \hat{B}_{i,m} \rrbracket\}_{i \in [N_{\text{valid}}], m \in [M]}$; shared scaled loads $\{\llbracket \tilde{W}_i \rrbracket\}_{i \in [N_{\text{valid}}]}$; shared global denominator $\llbracket \Delta \rrbracket$.
Output: Shared scores $\{(\llbracket \text{Score}_m^{\text{num}} \rrbracket, \llbracket \text{Score}_m^{\text{den}} \rrbracket)\}_{m \in [M]}$; shared counts and load sums $\{(\llbracket |A(m)| \rrbracket, \llbracket \tilde{T}_m \rrbracket)\}_{m \in [M]}$.

```

1 for  $m \leftarrow 1$  to  $M$  do
2    $\llbracket |A(m)| \rrbracket \leftarrow \llbracket 0 \rrbracket$ ;
3    $\llbracket \tilde{T}_m \rrbracket \leftarrow \llbracket 0 \rrbracket$ ;
4   for  $i \leftarrow 1$  to  $N_{\text{valid}}$  do
5      $\llbracket |A(m)| \rrbracket \leftarrow \llbracket |A(m)| \rrbracket + \llbracket \hat{B}_{i,m} \rrbracket$ ;
6      $\llbracket \text{contrib}_{i,m} \rrbracket \leftarrow \text{SecureMult}(\llbracket \hat{B}_{i,m} \rrbracket, \llbracket \tilde{W}_i \rrbracket)$ ;
7      $\llbracket \tilde{T}_m \rrbracket \leftarrow \llbracket \tilde{T}_m \rrbracket + \llbracket \text{contrib}_{i,m} \rrbracket$ ;
8    $\llbracket \text{Score}_m^{\text{num}} \rrbracket \leftarrow \llbracket \Delta \rrbracket - \llbracket \tilde{T}_m \rrbracket$ ;
9    $\llbracket \text{Score}_m^{\text{den}} \rrbracket \leftarrow \text{SecureMult}(\llbracket \Delta \rrbracket, \llbracket |A(m)| \rrbracket)$ ;

```

Remark 5.1 (Zero-support handling and assumption). *To ensure the correct execution and numerical stability of the protocol, we must consider cases where candidates receive zero support.*

First, we establish a global assumption: we assume that in every election period, at least one candidate receives at least one approval vote. The degenerate case where the ballot matrix is completely empty (i.e., $|A(m)| = 0$ for all m) is excluded, as it represents a void election and would cause the global denominator Δ to collapse to zero during the wallet update (Algorithm 7).

Second, we address what happens when some candidates (but not all) receive zero support ($|A(m)| = 0$). In Phragmén’s rule, a candidate with no supporters would theoretically require infinite time to be funded. The MPC implementation handles this case obliviously, without division by zero or special-case branching.

Because Algorithm 5 compares scores using cross-multiplication instead of actual division, a candidate m with zero supporters will always yield a score calculation that evaluates as mathematically strictly worse than any candidate m' with positive support ($|A(m')| > 0$). Specifically, the zero-support candidate effectively assumes the value of $+\infty$ and naturally loses the comparison.

This implicit handling keeps the algorithm data-oblivious.

By not explicitly checking for zeros via conditional branching, we keep the

control flow data-oblivious, with input-independent execution time, and avoid leaking any side-channel information regarding which candidates received zero support.

5.5. Find Minimum Score

Algorithm 5: Find Minimum Score.

Input: Shared scores $\{(\llbracket \text{Score}_m^{\text{num}} \rrbracket, \llbracket \text{Score}_m^{\text{den}} \rrbracket)\}_{m \in [M]}$.
Output: Shared winning score $(\llbracket \text{Score}^{*\text{num}} \rrbracket, \llbracket \text{Score}^{*\text{den}} \rrbracket)$.

- 1 $\llbracket \text{best}^{\text{num}} \rrbracket \leftarrow \llbracket \text{Score}_1^{\text{num}} \rrbracket;$
- 2 $\llbracket \text{best}^{\text{den}} \rrbracket \leftarrow \llbracket \text{Score}_1^{\text{den}} \rrbracket;$
- 3 **for** $m \leftarrow 2$ **to** M **do**
- 4 $\llbracket \text{cross}_a \rrbracket \leftarrow \text{SecureMult}(\llbracket \text{Score}_m^{\text{num}} \rrbracket, \llbracket \text{best}^{\text{den}} \rrbracket);$
- 5 $\llbracket \text{cross}_b \rrbracket \leftarrow \text{SecureMult}(\llbracket \text{best}^{\text{num}} \rrbracket, \llbracket \text{Score}_m^{\text{den}} \rrbracket);$
- 6 $\llbracket \beta_m \rrbracket \leftarrow \text{SecureLT}(\llbracket \text{cross}_a \rrbracket, \llbracket \text{cross}_b \rrbracket);$
- 7 $\llbracket \text{best}^{\text{num}} \rrbracket \leftarrow$
 $\llbracket \text{best}^{\text{num}} \rrbracket + \text{SecureMult}(\llbracket \beta_m \rrbracket, \llbracket \text{Score}_m^{\text{num}} \rrbracket - \llbracket \text{best}^{\text{num}} \rrbracket);$
- 8 $\llbracket \text{best}^{\text{den}} \rrbracket \leftarrow \llbracket \text{best}^{\text{den}} \rrbracket + \text{SecureMult}(\llbracket \beta_m \rrbracket, \llbracket \text{Score}_m^{\text{den}} \rrbracket - \llbracket \text{best}^{\text{den}} \rrbracket);$
- 9 $\llbracket \text{Score}^{*\text{num}} \rrbracket \leftarrow \llbracket \text{best}^{\text{num}} \rrbracket;$
- 10 $\llbracket \text{Score}^{*\text{den}} \rrbracket \leftarrow \llbracket \text{best}^{\text{den}} \rrbracket;$

5.6. One-Hot Winner Identification

To ensure fairness during tie-breaking, the protocol iterates over the candidates using the public permutation π generated in Algorithm 1.

Algorithm 6: One-Hot Winner with Permuted Tie-Break.

Input: Shared scores $\{(\llbracket \text{Score}_m^{\text{num}} \rrbracket, \llbracket \text{Score}_m^{\text{den}} \rrbracket)\}_{m \in [M]}$; shared winning score $(\llbracket \text{Score}^{*\text{num}} \rrbracket, \llbracket \text{Score}^{*\text{den}} \rrbracket)$; public permutation $\pi \in S_M$.

Output: Shared one-hot indicator $\{\llbracket \chi_m \rrbracket\}_{m \in [M]}$ with $\chi_{m^*} = 1$ for the unique winner m^* .

```
1 for  $m \leftarrow 1$  to  $M$  do
2    $\llbracket \alpha_m \rrbracket \leftarrow \text{SecureMult}(\llbracket \text{Score}_m^{\text{num}} \rrbracket, \llbracket \text{Score}^{*\text{den}} \rrbracket)$ ;
3    $\llbracket \beta_m \rrbracket \leftarrow \text{SecureMult}(\llbracket \text{Score}^{*\text{num}} \rrbracket, \llbracket \text{Score}_m^{\text{den}} \rrbracket)$ ;
4    $\llbracket e_m \rrbracket \leftarrow \text{SecureIsZero}(\llbracket \alpha_m \rrbracket - \llbracket \beta_m \rrbracket)$ ;
5    $m_1 \leftarrow \pi(1)$ ;
6    $\llbracket \chi_{m_1} \rrbracket \leftarrow \llbracket e_{m_1} \rrbracket$ ;
7    $\llbracket s \rrbracket \leftarrow \llbracket e_{m_1} \rrbracket$ ;
8   for  $k \leftarrow 2$  to  $M$  do
9      $m \leftarrow \pi(k)$ ;
10     $\llbracket \chi_m \rrbracket \leftarrow \text{SecureMult}(\llbracket e_m \rrbracket, \llbracket 1 \rrbracket - \llbracket s \rrbracket)$ ;
11     $\llbracket s \rrbracket \leftarrow \llbracket s \rrbracket + \llbracket \chi_m \rrbracket$ ;
```

5.7. Wallet Update

The wallets of the winning candidate's supporters are reset to 0, while the wallets of non-supporters are incremented by the winning score. By scaling the global denominator by $|A(m^*)|$, updates require zero divisions.

Algorithm 7: Wallet Update (Global Denominator Scaling).

Input: Shared ballot matrix $\{\llbracket \hat{B}_{i,m} \rrbracket\}_{i \in [N_{\text{valid}}], m \in [M]}$; shared one-hot winner $\{\llbracket \chi_m \rrbracket\}_{m \in [M]}$; shared current loads $\{\llbracket \tilde{W}_i \rrbracket\}_{i \in [N_{\text{valid}}]}$; shared global denominator $\llbracket \Delta \rrbracket$; shared candidates' data $\{(\llbracket A(m) \rrbracket, \llbracket \tilde{T}_m \rrbracket)\}_{m \in [M]}$.

Output: Shared updated loads $\{\llbracket \tilde{W}_i \rrbracket\}_{i \in [N_{\text{valid}}]}$ and new global denominator $\llbracket \Delta \rrbracket$.

```

1  $\llbracket A(m^*) \rrbracket \leftarrow \llbracket 0 \rrbracket$ ;
2  $\llbracket \tilde{T}^* \rrbracket \leftarrow \llbracket 0 \rrbracket$ ;
3 for  $m \leftarrow 1$  to  $M$  do
4    $\llbracket A(m^*) \rrbracket \leftarrow \llbracket A(m^*) \rrbracket + \text{SecureMult}(\llbracket \chi_m \rrbracket, \llbracket A(m) \rrbracket)$ ;
5    $\llbracket \tilde{T}^* \rrbracket \leftarrow \llbracket \tilde{T}^* \rrbracket + \text{SecureMult}(\llbracket \chi_m \rrbracket, \llbracket \tilde{T}_m \rrbracket)$ ;
6  $\llbracket \kappa^* \rrbracket \leftarrow \llbracket \Delta \rrbracket - \llbracket \tilde{T}^* \rrbracket$ ;
7 for  $i \leftarrow 1$  to  $N_{\text{valid}}$  do
8    $\llbracket \gamma_i \rrbracket \leftarrow \llbracket 0 \rrbracket$ ;
9   for  $m \leftarrow 1$  to  $M$  do
10     $\llbracket \gamma_i \rrbracket \leftarrow \llbracket \gamma_i \rrbracket + \text{SecureMult}(\llbracket \hat{B}_{i,m} \rrbracket, \llbracket \chi_m \rrbracket)$ ;
11     $\llbracket \mu_i \rrbracket \leftarrow \text{SecureMult}(\llbracket \tilde{W}_i \rrbracket, \llbracket A(m^*) \rrbracket)$ ;
12     $\llbracket \nu_i \rrbracket \leftarrow \llbracket \mu_i \rrbracket + \llbracket \kappa^* \rrbracket$ ;
13     $\llbracket \tilde{W}_i \rrbracket \leftarrow \text{SecureMult}(\llbracket 1 \rrbracket - \llbracket \gamma_i \rrbracket, \llbracket \nu_i \rrbracket)$ ;
14  $\llbracket \Delta \rrbracket \leftarrow \text{SecureMult}(\llbracket \Delta \rrbracket, \llbracket A(m^*) \rrbracket)$ ;
```

5.8. Winner Reconstruction

Algorithm 8: Winner Reconstruction.

Input: Shared one-hot indicator $\{\llbracket \chi_m \rrbracket\}_{m \in [M]}$.

Output: The elected candidate $m^* \in [M]$, revealed publicly.

```

1 for  $m \leftarrow 1$  to  $M$  do
2    $\chi_m \leftarrow \text{Reconstruct}_t(\llbracket \chi_m \rrbracket)$ ;
3   if  $\chi_m \notin \{0, 1\}$  then
4     abort;
5 if  $\sum_{m=1}^M \chi_m \neq 1$  then
6   abort;
7  $m^* \leftarrow$  the unique  $m \in [M]$  with  $\chi_m = 1$ ;
8 return  $m^*$ ;
```

6. Complexity Analysis

We analyse the cost of PP-PHRAGMÉN in the standard MPC cost model, separating three quantities. The *round complexity* is the number of sequential communication rounds, i.e. the depth of the dependency chain of secure operations; we treat each invocation of `SecureMult`, `SecureLT`, or `SecureZero` as a constant-round sub-protocol, since the multiplication of $[\cdot]$ and the comparison/equality tests of $[\cdot]$ are all constant-round. The *communication (message) complexity* is the total number of such secure-operation invocations; since each consumes $O(D)$ field elements per party, the bit-level communication is obtained by multiplying by $O(D\ell)$, where

$$\ell := \lceil \log_2 p \rceil$$

is the field bit-length. The *computation complexity* is the number of local operations performed by each party.

Two conventions are used throughout. First, mutually independent secure operations are *batched* into a single round; the round count therefore measures the longest sequential chain, not the raw operation count. Second, we separate *one-time setup* (Algorithms 1–3) from the *per-period loop* (Algorithms 4–8), which is repeated once per period $r \in [R]$. Algorithm 2 (validation) is charged to setup under the assumption that the approval profile is fixed across the horizon, as in the recurring-election model of Section 3; if voters resubmit ballots each period, its per-period cost is added to the loop, leaving all asymptotic totals below unchanged.

6.1. Per-Algorithm Analysis

Algorithm 1 (Oblivious Candidate Permutation, setup). This is a commit-then-reveal coin toss followed by a local Fisher–Yates shuffle. It uses *no* secure arithmetic. *Rounds:* $O(1)$ — one broadcast round for the D commitments and one for the D openings. *Communication:* $O(D)$ broadcasts (no `SecureMult`/`SecureLT`). *Computation:* $O(D)$ hash evaluations for verification and $O(M)$ local work for the permutation, per party.

Algorithm 2 (Validation, Sanitization & Compaction, setup). For every entry the protocol forms the product $B_{n,m}(B_{n,m} - 1)$ and reconstructs it. The NM products are mutually independent and are batched, as are the NM reconstructions; compaction is a purely local re-indexing of shares. *Rounds:* $O(1)$ (one multiplication layer, one reconstruction layer). *Communication:*

NM **SecureMult** and NM reconstructions, hence $O(MN)$. *Computation:* $O(MN)$ local. The literal pseudocode short-circuits on the first detected cheater; this is a local branch on a *revealed* value, so it affects only the best-case running time and never the worst case analysed here.

Algorithm 3 (Initialization, setup)... Both $\llbracket \Delta \rrbracket \leftarrow \llbracket 1 \rrbracket$ and the N assignments $\llbracket W_n \rrbracket \leftarrow \llbracket 0 \rrbracket$ are local sharings of public constants. *Rounds:* 0. *Communication:* 0. *Computation:* $O(N)$ local.

Algorithm 4 (Per-Candidate Score Computation)... For each candidate the protocol accumulates $|A(m)|$ and $\tilde{T}_m$ by affine (local) additions, computes one product $\text{contrib}_{i,m} = \text{SecureMult}(\tilde{B}_{i,m}, \tilde{W}_i)$ per voter, and one denominator product $\text{Score}_m^{\text{den}} = \text{SecureMult}(\Delta, |A(m)|)$ per candidate. All $MN + M$ products take disjoint, already-available shares as inputs and are therefore mutually independent. *Rounds:* $O(1)$ (a single batched multiplication layer). *Communication:* $MN + M = O(MN)$ **SecureMult**. *Computation:* $O(MN)$ local.

Algorithm 5 (Find Minimum Score)... This is the round-complexity bottleneck of the protocol. The reduction maintains a running best score ($\text{best}^{\text{num}}, \text{best}^{\text{den}}$) and, in iteration m , performs two cross-products $\text{cross}_a, \text{cross}_b$, one comparison $\beta_m = \text{SecureLT}(\text{cross}_a, \text{cross}_b)$, and two multiplexer updates of best^{num} and best^{den} . Crucially, $(\text{best}^{\text{num}}, \text{best}^{\text{den}})$ at iteration m depends on its value at iteration $m - 1$, so the $M - 1$ iterations form a *sequential chain* that cannot be batched. *Rounds:* $O(M)$; each iteration contributes a constant number of secure-operation layers, and there are $M - 1$ of them in series. *Communication:* $4(M - 1)$ **SecureMult** and $M - 1$ **SecureLT**, i.e. $O(M)$ of each. *Computation:* $O(M)$ local.

Remark 6.1 (The $O(M)$ round bottleneck). *The linear-depth reduction in Algorithm 5 is the dominant source of sequential rounds in the per-period loop: every other per-period algorithm runs in $O(1)$ rounds except the tie-break prefix scan of Algorithm 6 (below), which is likewise a sequential chain of length M . Both can be reduced to $O(\log M)$ rounds by replacing the left-to-right scan with a balanced binary tournament: the minimum is found by a tree of pairwise comparisons, and the first-match tie-break by a parallel-prefix (work-efficient scan) network. This lowers the per-period round complexity from $O(M)$ to $O(\log M)$ at the same asymptotic communication cost, and is the single most effective latency optimization for large candidate sets.*

Algorithm 6 (One-Hot Winner with Permuted Tie-Break).. The first loop computes, for each candidate, two cross-products and one equality test $e_m = \text{SecureIsZero}(\alpha_m - \beta_m)$; these M triples are mutually independent and are batched, costing $O(1)$ rounds and $2M$ **SecureMult** and M **SecureIsZero**. The second loop walks the candidates in the public order π and suppresses all but the first match via the running indicator $\llbracket s \rrbracket$; because s is a strict prefix accumulator, $\chi_{\pi(k)} = \text{SecureMult}(e_{\pi(k)}, 1 - s)$ depends on the preceding χ 's, forming a sequential chain of length $M - 1$. *Rounds:* $O(M)$ (dominated by the prefix scan; cf. Remark 6.1). *Communication:* $2M + (M - 1) = O(M)$ **SecureMult** and M **SecureIsZero**. *Computation:* $O(M)$ local.

Algorithm 7 (Wallet Update).. The first loop extracts the winner's data $|A(m^*)|, \tilde{T}^*$ via $2M$ independent products. The per-voter selector γ_i is the sum of M products $\text{SecureMult}(\hat{B}_{i,m}, \chi_m)$, contributing MN independent products; these are independent of the first loop and join the same round. The reset/increment then requires, per voter, one product $\mu_i = \text{SecureMult}(\tilde{W}_i, |A(m^*)|)$ (which depends on the first loop) and one product $\tilde{W}_i = \text{SecureMult}(1 - \gamma_i, \nu_i)$, plus the single denominator update $\Delta \leftarrow \text{SecureMult}(\Delta, |A(m^*)|)$. These form a constant number of dependent layers. *Rounds:* $O(1)$. *Communication:* $2M + MN + N + N + 1 = O(MN)$ **SecureMult**. *Computation:* $O(MN)$ local.

Algorithm 8 (Winner Reconstruction).. The M shares $\llbracket \chi_m \rrbracket$ are reconstructed (batched) and checked. *Rounds:* $O(1)$. *Communication:* M reconstructions; no **SecureMult**/**SecureLT**. *Computation:* $O(M)$ local (interpolation and the consistency check $\sum_m \chi_m = 1$).

6.2. End-to-End Complexity

Summing the per-period algorithms (Algorithms 4–8), a single election period costs

$$\underbrace{O(M)}_{\text{rounds}}, \quad \underbrace{O(MN)}_{\text{SecureMult}}, \quad \underbrace{O(M)}_{\text{SecureLT}}, \quad \underbrace{O(M)}_{\text{SecureIsZero}},$$

with $O(MN)$ local computation per party. The round count is governed by the sequential reductions of Remark 6.1, whereas the communication is governed by the two “dense” loops over the ballot matrix — score computation and wallet update — each of which issues $\Theta(MN)$ multiplications. Table 1 collects the per-algorithm figures.

Algorithm	Rounds	SecureMult	SecureLT	SecurelsZero	Local
1 Permutation [†]	$O(1)$	—	—	—	$O(M+D)$
2 Validation [†]	$O(1)$	$O(MN)$	—	—	$O(MN)$
3 Initialization [†]	$O(1)$	—	—	—	$O(N)$
4 Score	$O(1)$	$O(MN)$	—	—	$O(MN)$
5 Find-Min	$O(\mathbf{M})$	$O(M)$	$O(M)$	—	$O(M)$
6 One-Hot	$O(\mathbf{M})$	$O(M)$	—	$O(M)$	$O(M)$
7 Wallet Update	$O(1)$	$O(MN)$	—	—	$O(MN)$
8 Reveal	$O(1)$	—	—	—	$O(M)$
Per period ($\times R$)	$O(M)$	$O(MN)$	$O(M)$	$O(M)$	$O(MN)$
End-to-end	$O(RM)$	$O(RMN)$	$O(RM)$	$O(RM)$	$O(RMN)$

Table 1: Asymptotic complexity of PP-PHRAGMÉN. Each secure operation is constant-round; “Rounds” counts sequential layers. Boldface marks the $O(M)$ sequential reductions (Remark 6.1). Bit-level communication is obtained by multiplying the operation counts by $O(D\ell)$, where comparison and equality tests additionally carry an internal $O(\ell)$ multiplication cost (see Remark 6.2). [†]One-time setup; not multiplied by R .

Over the full horizon of R periods, the one-time setup (Algorithms 1–3, total $O(MN)$ work in $O(1)$ rounds) is dominated, giving the end-to-end totals in the last row of Table 1:

$$\text{rounds } O(RM), \quad \text{secure multiplications } O(RMN), \quad \text{comparisons/equalities } O(RM). \quad (5)$$

Remark 6.2 (Bit-level cost of comparisons). *Each SecureLT and SecurelsZero invocation internally consumes $O(\ell)$ multiplications, where $\ell = \lceil \log_2 p \rceil$. The operation counts in Table 1 therefore translate to a bit-level communication of $O(RMN D \ell)$ for the dense loops and $O(RM D \ell^2)$ for the comparison chains. This makes the dependence of the cost on ℓ explicit, and links directly to Section 7: since $\ell = \Omega(R \log N)$ is forced by the denominator growth, the comparison sub-protocols become progressively more expensive over the horizon, and the bit-level cost of the protocol is super-linear in R .*

7. Field Size Requirements and Boundedness

All arithmetic is carried out in $\mathbb{F}_p$, yet the protocol reasons about the *rational* balances $w_n \in \mathbb{Q}_{\geq 0}$ through the invariant

$$w_n = \frac{\tilde{W}_n}{\Delta}, \quad \tilde{W}_n, \Delta \in \mathbb{F}_p, \Delta > 0, \quad (6)$$

maintained at all times by Algorithms 3 and 7. Two facts must be established for the protocol to be correct over a horizon of R periods: (i) that the integers fed to `SecureLT` and `SecurelsZero` are non-negative and do not wrap around modulo p , which we obtain from a boundedness lemma on the support sums; and (ii) an explicit lower bound on ℓ that guarantees no wrap-around occurs through period R . We treat the two in turn.

7.1. Growth of the Global Denominator

Recall from Algorithm 7 that, after the winner m^* of period r is selected, the denominator is updated as $\Delta \leftarrow \Delta \cdot |A(m^*)|$. Writing Δ_r for the value of Δ at the *start* of period r (i.e. the value used in Algorithm 4 during period r), Algorithm 3 sets $\Delta_1 = 1$ and the recurrence is

$$\Delta_{r+1} = \Delta_r \cdot |A(m_r^*)|, \quad \text{hence} \quad \Delta_r = \prod_{k=1}^{r-1} |A(m_k^*)|, \quad (7)$$

with the empty product equal to 1 for $r = 1$. This scaling is not incidental: it is exactly the factor required to keep all wallets over a common integer denominator after the division-free update, as Equation (6) shows. Since $1 \leq |A(m_k^*)| \leq N_{\text{valid}} \leq N$, we obtain the two-sided bound

$$1 \leq \Delta_r \leq N^{r-1}, \quad (8)$$

and whenever every elected candidate has at least two supporters the lower bound improves to $\Delta_r \geq 2^{r-1}$. Thus Δ_r grows *exponentially in the period index r* ; this is the central quantity driving the field-size requirement.

7.2. Wallet Boundedness

The well-definedness of every comparison in the protocol rests on the support sums staying within $[0, \Delta_r]$, equivalently on the rational support sums staying within $[0, 1]$.

Lemma 7.1 (Wallet boundedness). *For every period $r \in [R]$ and every candidate $m \in [M]$, the support sum at the start of period r satisfies*

$$0 \leq \sum_{n \in A(m)} w_n^{(r)} \leq 1, \quad \text{equivalently} \quad 0 \leq \tilde{T}_m^{(r)} \leq \Delta_r,$$

where $\tilde{T}_m^{(r)} = \sum_{n \in A(m)} \tilde{W}_n^{(r)}$ and the equivalence follows from Equation (6).

Proof. Non-negativity is immediate: by Algorithms 3 and 7 every wallet is either reset to 0 or incremented by the winning score $\text{Score}^* \geq 0$, so $w_n^{(r)} \geq 0$ for all n, r ; summing over $A(m)$ gives $\tilde{T}_m^{(r)} \geq 0$. It remains to prove the upper bound, by induction on r .

Base case ($r = 1$). All wallets are initialized to 0 (Algorithm 3), so $\sum_{n \in A(m)} w_n^{(1)} = 0 \leq 1$ for every m .

Inductive step. Assume the bound holds at the start of period r . Let m^* be the candidate elected in period r . By the hypothesis applied to m^* , $\sum_{n \in A(m^*)} w_n^{(r)} \leq 1$, so the winning score

$$\text{Score}^* = \frac{1 - \sum_{n \in A(m^*)} w_n^{(r)}}{|A(m^*)|} \geq 0,$$

which also confirms that the numerator $\text{Score}_{m^*}^{\text{num}} = \Delta_r - \tilde{T}_{m^*}^{(r)} \geq 0$ is non-negative, as required for the comparison to be meaningful.

Now fix an arbitrary candidate m and split its supporters according to whether they also support the winner. By Equation (2), supporters in $A(m) \cap A(m^*)$ are reset to 0, while those in $A(m) \setminus A(m^*)$ are incremented by Score^* . Writing $a := \sum_{n \in A(m) \setminus A(m^*)} w_n^{(r)} \geq 0$ and $c := |A(m) \setminus A(m^*)| \leq |A(m)|$, the support sum at the start of period $r + 1$ is

$$\sum_{n \in A(m)} w_n^{(r+1)} = a + c \cdot \text{Score}^*.$$

The arg-min property of m^* (Equation (1)) gives, for the candidate m ,

$$|A(m)| \cdot \text{Score}^* \leq 1 - \sum_{n \in A(m)} w_n^{(r)} = 1 - (a + b), \quad b := \sum_{n \in A(m) \cap A(m^*)} w_n^{(r)} \geq 0,$$

and by the induction hypothesis the right-hand side $1 - (a + b) \geq 0$. Therefore, using $c \leq |A(m)|$,

$$\sum_{n \in A(m)} w_n^{(r+1)} = a + c \cdot \text{Score}^* \leq a + |A(m)| \cdot \text{Score}^* \leq a + (1 - a - b) = 1 - b \leq 1.$$

The bound is thus preserved, completing the induction. Note that the argument also covers the “already-fundable-at-start” branch of the rule: if some candidate has support sum exactly 1 at the start of a period, then $\text{Score}^* = 0$, every non-reset wallet is unchanged, and the bound is trivially preserved. $\square$

Corollary 7.2 (Well-definedness of the secure comparisons). *At the start of any period r , the score operands satisfy*

$$0 \leq \text{Score}_m^{\text{num}} = \Delta_r - \tilde{T}_m^{(r)} \leq \Delta_r, \quad 0 \leq \text{Score}_m^{\text{den}} = |A(m)| \cdot \Delta_r \leq N \Delta_r,$$

so the cross-products compared by SecureLT in Algorithm 5 and by SecureIsZero in Algorithm 6 are non-negative integers bounded by

$$V_r := \max_m (\text{Score}_m^{\text{num}} \cdot \text{Score}_{m'}^{\text{den}}) \leq N \Delta_r^2.$$

Consequently, every comparison and equality test in period r is well-defined — it correctly reflects the order of the underlying rationals and induces no modular wrap-around — provided V_r lies below the wrap-around threshold of the field, which we make precise next.

7.3. Field-Size Requirement

Let κ be the statistical security parameter of the comparison/equality sub-protocols, which require their inputs to lie in a public range of width $2^{\ell-\kappa}$ strictly below p to avoid both arithmetic and masking overflow. Writing $V_{\max} = \max_{r \in [R]} V_r$ for the largest integer ever fed to a comparison over the horizon, the field must satisfy

$$V_{\max} < 2^{\ell-\kappa} \leq \frac{p}{2}, \quad \text{i.e.} \quad \ell \geq \log_2 V_{\max} + \kappa + O(1). \quad (9)$$

First period ($r = 1$).. By Equation (8), $\Delta_1 = 1$, so all wallets are 0 and the scores reduce to $\text{Score}_m^{\text{num}} = 1$, $\text{Score}_m^{\text{den}} = |A(m)| \leq N$. The cross-products compared in Algorithm 5 are then simply the supporter counts, $V_1 \leq N$. Hence the first period alone imposes the mild requirement

$$\ell \geq \log_2 N + \kappa + O(1), \quad \text{i.e.} \quad \ell = \Omega(\log N). \quad (10)$$

A field accommodating a few times N therefore suffices to run the protocol *for one period*; the difficulty is entirely a consequence of *persistence* across periods.

Subsequent periods ($r > 1$).. Combining Theorem 7.2 with the denominator bound (8),

$$V_r \leq N \Delta_r^2 \leq N \cdot (N^{r-1})^2 = N^{2r-1}, \quad (11)$$

so to guarantee correctness *through period* r the field-length must grow as

$$\ell \geq (2r - 1) \log_2 N + \kappa + O(1), \quad \text{i.e.} \quad \ell = \Omega(r \log N). \quad (12)$$

The same lower bound applies to the largest intermediate value of the wallet update: for a voter who supports no elected candidate, the scaled wallet may reach $\tilde{W}_i \leq r \Delta_r$, and the products $\mu_i = \tilde{W}_i \cdot |A(m^*)|$ and $\Delta_{r+1} = \Delta_r \cdot |A(m^*)|$ are then bounded by $N r \Delta_r$, which is dominated by $V_r = \Theta(N \Delta_r^2)$ whenever $\Delta_r \geq r$. Thus the comparison cross-products are the binding constraint.

Corollary 7.3 (Horizon field-size requirement). *To execute PP-PHRAGMÉN correctly over an entire horizon of R periods with a single fixed prime p , it suffices — and, up to the additive security term, is necessary — that*

$$\ell = \lceil \log_2 p \rceil \geq (2R - 1) \log_2 N + \kappa + O(1), \quad \text{i.e.} \quad \log_2 p = \Omega(R \log N).$$

(13)

Equivalently, p must exceed N^{2R-1} by a factor of $2^{\kappa+O(1)}$. The field size therefore grows linearly in the horizon R and logarithmically in the electorate size N .

Proof. Take $r = R$ in (11) to obtain $V_{\max} \leq N^{2R-1}$, and substitute into (9). Necessity follows because, by the improved lower bound $\Delta_R \geq 2^{R-1}$ whenever winners have at least two supporters, $V_R = \Theta(N \Delta_R^2)$ can be as large as $\Theta(N \cdot 4^{R-1})$, forcing $\ell = \Omega(R)$ in that regime and, more generally, $\ell = \Omega(R \log N)$ when winner support counts are $\Theta(N)$. $\square$

Remark 7.1 (Renormalization and the cost of persistence). *Theorem 7.3 quantifies the price of the division-free design: the protocol trades the per-voter denominator blow-up of the naive representation (which would force $\log p = \Omega(N)$ within a single period) for a global denominator that grows by one factor of $|A(m^*)|$ per period, hence $\log p = \Omega(R \log N)$ across the horizon. The growth can in principle be curbed by periodically renormalizing, i.e. dividing Δ and all $\tilde{W}_n$ by a common integer factor so as to restore a small denominator. This is delicate: after the update, $\tilde{W}_i \leftarrow \tilde{W}_i |A(m^*)| + \kappa^*$ for non-supporters, and the additive term $\kappa^* = \Delta - \tilde{T}^*$ generally destroys divisibility by $|A(m^*)|$, so no fixed renormalizing factor exists. A correct renormalization must extract the common factor obliviously — e.g. via a secure*

GCD or a secure division by the largest denominator that divides every $\tilde{W}_n$ — at a substantial additional MPC cost. Characterizing the smallest stable denominator that can be maintained (a structural floor below which renormalization cannot push Δ without loss of precision) is a natural direction for a perpetual-maintenance variant of the protocol, and we leave a quantitative treatment to future work.